

GenAI Engineering Learning Path

From "How LLMs Work" to Building Real Systems (2026 Edition)

This document starts from your slide deck (tokenization → next-token prediction → autoregressive generation) and builds it out into a full learning path for GenAI engineering work. Each section has: **what it is, why it matters, how deep to go, and what's practical.**

Depth key used throughout:

- **Must know deeply** — you'll use this weekly, need real intuition
- **Know how it works, don't need to build it** — conceptual fluency is enough
- **Skip or learn later** — interesting but not blocking for engineering work

Phase 0: The Core Idea (Your Slides — Deepened)

This is the foundation everything else sits on. Your deck got the shape right; here's the missing depth.

● 1. Tokens aren't words

Your slide showed "ChatGPT is amazing!" → ["Chat", "GPT", " is", " amazing", "!"].

Here's what matters practically:

- Tokenization is done by an algorithm called **Byte Pair Encoding (BPE)** (or variants like SentencePiece). It's trained separately from the model itself, on a large text corpus, to find the most common recurring chunks of characters.
- Why this matters day-to-day:** token count = your cost and your context limit. "500 words" is not "500 tokens" — English averages roughly 0.75 words per token, so 500 words ≈ 650–700 tokens.
- Uncommon words, code, non-English languages, and emojis tokenize *less efficiently* (more tokens per character). This is why non-English prompts often cost more and fit less into the same context window.
- Practical skill:** know how to count tokens before you ship a feature. For OpenAI models, the library is `tiktoken`. For Claude, Anthropic exposes a token-counting endpoint. You should never guess token counts in production — you count them.

```
python
```

```
import tiktoken
enc = tiktoken.encoding_for_model("gpt-4o")
print(len(enc.encode("ChatGPT is amazing!"))) # -> token count
```

● 2. Next-token prediction is the *only* thing the model does

Your slide's core insight is correct and important: **the model has one job** — given the tokens so far, output a probability distribution over what token comes next. Everything (chat, code, reasoning, agents) is this one operation repeated.

What to internalize as an engineer: **the model is not "thinking" in a separate step and then writing**. The words you see appearing are the reasoning — there's no hidden "answer" it already knows and is just typing out. This is *why* techniques like "chain-of-thought" (asking the model to reason step-by-step in the output) actually improve accuracy: the model gets to use its own earlier tokens as working memory. This is not a trick — it's a direct consequence of the autoregressive mechanism your slides describe.

● 3. Probabilities and sampling (your slide showed this, but stopped short)

Your deck showed: Paris → 0.85, London → 0.05, Berlin → 0.03. The missing piece: **how does the model pick one?**

This is controlled by parameters you will tune constantly:

Parameter	What it does	Practical guidance
Temperature	Controls randomness. 0 = always pick highest probability (deterministic-ish). Higher (0.7–1.0) = more variety/creativity.	Use low temp (0–0.3) for factual/structured tasks (extraction, code, classification). Use higher (0.7+) for creative writing/brainstorming.
Top-p (nucleus sampling)	Only consider tokens whose cumulative probability adds up to p (e.g., 0.9). Cuts off the unlikely long tail (like "Pizza → 0.0001" in your slide).	Usually leave at default (0.9–1) unless you have a specific reason.
Top-k	Only consider the top k most likely tokens.	Rarely tuned directly in modern APIs; good to know it exists.

Why this matters: "the model is being random/inconsistent" is one of the most common production bugs, and it's almost always a temperature setting, not a model limitation.

● 4. It's not really "predict one word" — it's "predict one word, re-read everything, repeat"

Your slide's "Why the Output Becomes a Full Paragraph" example is the most important diagram in the deck. The engineering implication:

- Every single token costs a full forward pass through the model. This is **why longer outputs are slower and more expensive** — generation is inherently sequential, not

parallel.

- This is also why **latency scales with output length**, not input length (input is processed in parallel in one pass; output is generated one token at a time).
- **Practical skill:** when designing a product, always ask "does this need a long generated answer, or can I constrain the output?" Shorter expected outputs = faster, cheaper, more predictable products.

○ What your slides correctly chose to skip (and you can too, for now)

- The actual neural network math (attention weights, matrix multiplication, backpropagation) — not needed to build products.
- How embeddings are computed inside the model — needed only if you go into model training/fine-tuning research.

Phase 1: What's Missing From the Slides (Fill These Gaps)

These are the concepts every GenAI engineer needs that a "how it works" intro deck typically doesn't cover.

● 5. The Transformer & Attention (intuition only, not math)

You don't need the math, but you need the *idea*, because it explains almost every practical limitation you'll hit:

- **Self-attention** = when predicting the next token, the model looks back at *all* previous tokens and decides which ones matter most for this prediction. That's how "it" in a sentence can correctly refer back to a noun mentioned 200 words earlier.
- **Why this matters practically:** this is exactly why **context window size** is a hard engineering constraint. Every token in the context has to be attended to relative to every other token — this is computationally expensive, which is why longer contexts cost more and (historically) models get less accurate as context fills up ("lost in the middle" effect — models pay less attention to information buried in the middle of a long context vs. the start/end).
- **Practical implication:** don't just dump everything into context because you can. Put the most important information near the start or end of the prompt.

● 6. Context window & memory

- The context window is the model's *entire* working memory — there is no other memory unless you build it yourself (see RAG below).
- Every message in a "conversation" is really the *entire history* being re-sent as one long input each time. The model has no persistent memory between API calls by default.
- **Practical skill:** understand context window limits for the model you're using (varies by model, roughly 128K-1M+ tokens as of 2026), and design for what happens when a conversation exceeds it (truncation, summarization, or retrieval).

● 7. Embeddings (separate from tokenization — commonly confused)

- Tokenization turns text into IDs. **Embeddings** turn tokens (or whole chunks of text) into vectors of numbers that capture *meaning*, such that similar meanings are numerically close together.
- This is the backbone of **semantic search**, **RAG**, and **recommendation-style GenAI features**.
- **Practical skill**: know how to generate embeddings (`text-embedding-3-small`), Claude's embedding-compatible partners, or open-source options like `bge`/`e5` models) and store them in a vector database (Pinecone, Weaviate, pgvector, Chroma).

● 8. RAG — Retrieval-Augmented Generation

This is the single most common real-world GenAI architecture, and it's not in your slides at all.

- **The problem it solves**: the model's knowledge is frozen at training time and it can't access your private/current data.
- **How it works**:
 1. Break your documents into chunks
 2. Embed each chunk into a vector
 3. Store vectors in a vector database
 4. At query time, embed the user's question, find the most similar chunks, and stuff them into the prompt as context
 5. The model then answers using next-token prediction — but now grounded in the retrieved text
- **Why it matters**: this is how most production chatbots, internal knowledge assistants, and document Q&A tools work.
- **Depth needed**: you must be able to build a basic RAG pipeline end-to-end. Advanced RAG (re-ranking, hybrid search, query rewriting) can be learned when a specific project needs it.

● 9. Fine-tuning vs. Prompting vs. RAG (the decision every engineer must be able to make)

Approach	When to use	Cost/Effort
Prompting / few-shot examples	Default first choice, almost always	Low
RAG	Need up-to-date or private/proprietary information	Medium
Fine-tuning	Need consistent style/format/behavior baked in, or reducing prompt length for scale	High

In 2026, most production GenAI work is prompting + RAG. Fine-tuning is used far less often than beginners assume — know it exists, know when it's justified, don't default to it.

● 10. Structured output & tool calling (critical for real engineering)

- Modern LLMs can be constrained to output valid JSON matching a schema, or to call external "tools"/functions (search the web, query a database, run code) instead of just generating text.
- This is the mechanism behind **agents**: the model doesn't just chat, it decides "I should call this function," receives the result as new input tokens, and continues generating — same next-token-prediction loop, just with tool results injected into context.
- **Practical skill**: you need to be comfortable defining a tool schema and handling the request/response loop. This is core to building anything beyond a simple chatbot in 2026.

● 11. Agents & multi-step reasoning

- An "agent" is just an LLM in a loop: think → act (call a tool) → observe result → think again → ... until done.
- Know the pattern well enough to build a simple one. Don't over-invest in the many competing agent frameworks — the underlying loop is simple and frameworks change fast.

● 12. Evaluation ("evals")

- The most underrated skill for GenAI engineers. Because outputs are probabilistic, you cannot just "test it once and ship it."
- You need a repeatable way to measure: is this prompt/model/pipeline actually good? This means building a test set of realistic inputs with expected outcomes, and scoring outputs (automated checks, or an LLM-as-judge).
- Without evals, you're flying blind when you change a prompt, swap a model, or update your RAG pipeline.

● 13. Guardrails, safety, and failure modes

- **Hallucination:** the model will confidently generate plausible-sounding but false information — this is a *direct consequence* of next-token prediction (it's always predicting "what sounds right," not "what is true"). Mitigate with RAG (grounding), lower temperature, and asking the model to cite sources.
 - **Prompt injection:** if you're building RAG or agents, malicious text in retrieved documents or user input can try to hijack the model's instructions. Know this risk exists before you ship anything that reads external content.
 - **Cost/latency limits:** real products need timeouts, retries, and rate limit handling.
-

Phase 2: Practical Implementation Skills

This is the "actually build things" layer.

● Must-have hands-on skills

1. **Calling an LLM API** (Anthropic, OpenAI, or open-source via a hosting provider) — request/response, streaming, error handling.
2. **Prompt engineering** — system prompts, few-shot examples, structured formatting, iterating based on real outputs (not vibes).
3. **Building a basic RAG pipeline** — chunking, embedding, vector search, context assembly.
4. **Tool/function calling** — defining schemas, handling the call-result loop.
5. **Token & cost management** — counting tokens, estimating cost, setting max output limits.
6. **Basic evals** — a small test set + a scoring method, run before/after any change.

● Good to know, use as needed

- Vector database tradeoffs (Pinecone vs pgvector vs Chroma — mostly "pick one that fits your stack," not a deep specialization).
- Prompt caching (reduces cost/latency for repeated large contexts — increasingly standard in 2026 APIs).
- Streaming responses to improve perceived latency in UI.
- Multi-model routing (using a cheap/fast model for simple tasks, a stronger model for hard ones).

○ Learn later, only if your role needs it

- Training a model from scratch.
- Fine-tuning infrastructure (LoRA, quantization, GPU training pipelines) — only needed if you're on a model-training team, not a typical "build GenAI products" role.

- Deep transformer architecture math — needed for research roles, not engineering roles.

Suggested Order to Learn This (Roadmap)

```
Week 1-2    Phase 0 (tokens, next-token prediction, sampling params)
            + call an API directly, play with temperature
Week 3      Embeddings + vector search (build a tiny semantic search demo)
Week 4-5    RAG pipeline end-to-end (small doc set → chatbot that cites
            sources)
Week 6      Structured output + tool calling (build one simple tool-using
            agent)
Week 7      Evals (build a 20-example test set, score two prompt versions
            against it)
Week 8+     Pick a real project; learn fine-tuning, advanced RAG, guardrails
            only when the project actually demands them
```

What "Enough" Looks Like for a GenAI Engineer in 2026

You're ready for real engineering work when you can, without looking things up mid-task:

- Explain why an LLM sometimes gives inconsistent answers (temperature/sampling), and why it sometimes states false things confidently (hallucination from next-token prediction, not "lying").
- Build a RAG pipeline from raw documents to a grounded chatbot answer.
- Design and use a tool-calling loop for an agent.
- Estimate token cost and latency for a feature before building it.
- Set up a minimal eval so you can tell if a prompt change made things better or worse.

Everything past this (fine-tuning infra, novel agent frameworks, model training) is specialization you pick up **on the job, per project** — not prerequisite knowledge.